

LEAST SQUARES REGRESSION

16

CONTENTS

16.1	Least Squares Regression Problem Statement	279
16.2	Least Squares Regression Derivation (Linear Algebra)	280
16.3	Least Squares Regression Derivation (Multivariate Calculus)	281
16.4	Least Squares Regression in Python	283
16.4.1	Using the Direct Inverse Method	283
16.4.2	Using the Pseudo-Inverse	285
16.4.3	Using <code>numpy.linalg.lstsq</code>	285
16.4.4	Using <code>optimize.curve_fit</code> From SciPy	285
16.5	Least Squares Regression for Nonlinear Functions	286
16.5.1	Log Tricks for Exponential Functions	286
16.5.2	Log Tricks for Power Functions	288
16.5.3	Polynomial Regression	288
16.5.4	Using <code>optimize.curve_fit</code> From SciPy	289
16.6	Summary and Problems	290
16.6.1	Summary	290
16.6.2	Problems	291

16.1 LEAST SQUARES REGRESSION PROBLEM STATEMENT

Given a set of independent data points x_i and dependent data points $y_i, i = 1, \dots, m$, we would like to find an **estimation function**, $\hat{y}(x)$, that describes the data as accurately as possible. Note that $\hat{y}$ can be a function of several variables, but for the sake of this discussion, we restrict the domain of $\hat{y}$ to a single variable. In least squares regression, the estimation function must be a linear combination of **basis functions**, $f_i(x)$. That is, the estimation function must be of the form

$$\hat{y}(x) = \sum_{i=1}^n \alpha_i f_i(x).$$

The scalars α_i are referred to as the **parameters** of the estimation function, and each basis function must be linearly independent from the others. In other words, in the proper “functional space,” no basis function should be expressible as a linear combination of the other functions. Note that, in general, there are significantly more data points, m , than basis functions, n (i.e., $m \gg n$).

TRY IT! Create an estimation function for the force–displacement relationship of a linear spring. Identify the basis function(s) and model parameters.

The relationship between the force, F , and the displacement, x , can be described by the function $F(x) = kx$, where k is the spring stiffness. The only basis function is the function $f_1(x) = x$ and the model parameter to determine is $\alpha_1 = k$.

The goal of **least squares regression** is to find the parameters of the estimation function that minimize the **total squared error**, E , defined by $E = \sum_{i=1}^m (\hat{y}_i - y_i)^2$. The **individual errors** or **residuals** are defined as $e_i = (\hat{y}_i - y_i)$. If e is the vector containing all the individual errors, then we are also trying to minimize $E = \|e\|_2^2$, which is the L_2 norm defined in the previous chapter.

In the next two sections, we will derive the least squares method of finding the desired parameters. The first derivation comes from linear algebra, and the second comes from multivariate calculus. Although they are different derivations, they lead to the same least squares formula. You are free to focus on the section with which you are most comfortable.

16.2 LEAST SQUARES REGRESSION DERIVATION (LINEAR ALGEBRA)

First, we enumerate the estimation of the data at each data point x_i as

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\vdots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

Let $X \in \mathbb{R}^n$ be a column vector such that the i th element of X contains the value of the i th x -data point, x_i . $\hat{Y}$ be a column vector with elements, $\hat{Y}_i = \hat{y}(x_i)$, β be a column vector such that $\beta_i = \alpha_i$, $F_i(x)$ be a function that returns a column vector of $f_i(x)$ computed on every element of x , and A be an $m \times n$ matrix such that the i th column of A is $F_i(x)$. Given this notation, the previous system of equations becomes $\hat{Y} = A\beta$.

Now if Y is a column vector such that $Y_i = y_i$, the total squared error is given by $E = \|\hat{Y} - Y\|_2^2$. Verify this by substituting the definition of the L_2 norm. Since we want to make E as small as possible and norms are a measure of distance, this previous expression is equivalent to saying that we want $\hat{Y}$ and Y to be as “close as possible.” Note that, in general, Y will not be in the range of A and therefore $E > 0$.

Consider the following simplified depiction of the range of A ; see Fig. 16.1. Note this is *not* a plot of the data points (x_i, y_i) .

From observation, the vector in the range of A , namely $\hat{Y}$ which is closest to Y , is that which can point perpendicularly to Y ; therefore, we want a vector $Y - \hat{Y}$ that is perpendicular to the vector $\hat{Y}$.

Recall from linear algebra that two vectors are perpendicular, or orthogonal, if their dot-product is 0. Noting that the dot-product between two vectors, v and w , can be written as $\text{dot}(v, w) = v^T w$, we can state that $\hat{Y}$ and $Y - \hat{Y}$ are perpendicular if $\text{dot}(\hat{Y}, Y - \hat{Y}) = 0$; therefore, $\hat{Y}^T (Y - \hat{Y}) = 0$, which is equivalent to $(A\beta)^T (Y - A\beta) = 0$.

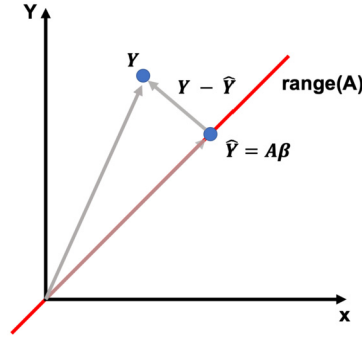


FIGURE 16.1

Illustration of the L_2 projection of Y on the range of A .

Noting that for the two matrices A and B we have $(AB)^T = B^T A^T$ and using distributive properties of vector multiplication, this is equivalent to $\beta^T A^T Y - \beta^T A^T A \beta = \beta^T (A^T Y - A^T A \beta) = 0$. The solution, $\beta = \mathbf{0}$, is a trivial solution, so we use $A^T Y - A^T A \beta = 0$ to find a more interesting solution. Solving this equation for β gives the **least squares regression formula**:

$$\beta = (A^T A)^{-1} A^T Y.$$

Note that $(A^T A)^{-1} A^T$ is called the **pseudo-inverse** of A and it exists when $m > n$ and A has linearly independent columns. Proving the invertibility of $(A^T A)$ is outside the scope of this book, but it is always invertible except for some specific cases.

16.3 LEAST SQUARES REGRESSION DERIVATION (MULTIVARIATE CALCULUS)

Recall that the total error for m data points and n basis functions is:

$$E = \sum_{i=1}^m e_i^2 = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2 = \sum_{i=1}^m \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right)^2,$$

which is an n -dimensional paraboloid in α_k . From calculus, we know that the minimum of a paraboloid is where all the partial derivatives equal zero. Taking the partial derivative of E with respect to the variable α_k (remember that in this case the parameters are our variables) and setting these derivatives equal to 0, and then solving the system of equations for the α_k 's should give the correct results.

Computing the partial derivative with respect to α_k and setting it equal to zero yields

$$\frac{\partial E}{\partial \alpha_k} = \sum_{i=1}^m 2 \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right) f_k(x_i) = 0.$$

With some rearrangement, the previous expression can be manipulated as follows:

$$\sum_{i=1}^m \sum_{j=1}^n \alpha_j f_j(x_i) f_k(x_i) - \sum_{i=1}^m y_i f_k(x_i) = 0,$$

and upon further rearrangement (we take advantage of the fact that addition commutes) the result is

$$\sum_{j=1}^n \alpha_j \sum_{i=1}^m f_j(x_i) f_k(x_i) = \sum_{i=1}^m y_i f_k(x_i).$$

Now let X be a column vector such that the i th element of X is x_i and Y similarly constructed, and let $F_j(X)$ be a column vector such that the i th element of $F_j(X)$ is $f_j(x_i)$. Using this notation, the previous expression can be rewritten in vector notation as

$$\begin{bmatrix} F_1^T(X)F_1(X), F_1^T(X)F_2(X), \dots, F_1^T(X)F_j(X), \dots, F_1^T(X)F_n(X) \\ F_2^T(X)F_1(X), F_2^T(X)F_2(X), \dots, F_2^T(X)F_j(X), \dots, F_2^T(X)F_n(X) \\ \vdots \\ F_n^T(X)F_1(X), F_n^T(X)F_2(X), \dots, F_n^T(X)F_j(X), \dots, F_n^T(X)F_n(X) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_j \\ \vdots \\ \alpha_n \end{bmatrix} = F_k^T(X)Y.$$

If we repeat this equation for every k , we get the following system of linear equations in matrix form:

$$\begin{bmatrix} F_1^T(X)F_1(X), F_1^T(X)F_2(X), \dots, F_1^T(X)F_j(X), \dots, F_1^T(X)F_n(X) \\ F_2^T(X)F_1(X), F_2^T(X)F_2(X), \dots, F_2^T(X)F_j(X), \dots, F_2^T(X)F_n(X) \\ \vdots \\ F_n^T(X)F_1(X), F_n^T(X)F_2(X), \dots, F_n^T(X)F_j(X), \dots, F_n^T(X)F_n(X) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_j \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} F_1^T(X)Y \\ F_2^T(X)Y \\ \vdots \\ F_n^T(X)Y \end{bmatrix}.$$

If we let $A = [F_1(X), F_2(X), \dots, F_j(X), \dots, F_n(X)]$ and β be a column vector such that the j th element of β is α_j , then the previous system of equations becomes

$$A^T A \beta = A^T Y,$$

and then solving this matrix equation for β gives $\beta = (A^T A)^{-1} A^T Y$, which is exactly the same formula as the previous derivation.

16.4 LEAST SQUARES REGRESSION IN PYTHON

Recall that enumerating the estimation of the data at each data point, x_i , will give us the following system of equations:

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\dots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

If the data were absolutely perfect (i.e., no noise), then the estimation function would go through all the data points, resulting in the following system of equations:

$$\begin{aligned}y_1 &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ y_2 &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\dots \\ y_m &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

If we were to take A as defined previously, this would result in the matrix equation

$$Y = A\beta.$$

Because the data is not perfect, there will not be an estimation function that can go through all the data points, and this system will have *no solution*. Therefore, we need to use the least square regression that we derived in the previous two sections to obtain a solution:

$$\beta = (A^T A)^{-1} A^T Y.$$

TRY IT! Consider the artificial data created by `x = np.linspace(0, 1, 101)` and `y = 1 + x + x * np.random.random(len(x))`. Do a least squares regression with an estimation function defined by $\hat{y} = \alpha_1 x + \alpha_2$. Plot the data points along with the least squares regression. Note that we expect $\alpha_1 = 1.5$ and $\alpha_2 = 1.0$ based on this data. Due to the random noise we added into the data, your results maybe slightly different. In the next few subsections, we will see how we solve this problem using different approaches.

16.4.1 USING THE DIRECT INVERSE METHOD

```
In [1]: import numpy as np
        from scipy import optimize
        import matplotlib.pyplot as plt

        plt.style.use("seaborn-poster")
```

```

In [2]: # generate x and y
        x = np.linspace(0, 1, 101)
        y = 1 + x + x * np.random.random(len(x))

In [3]: # assemble matrix A
        A = np.vstack([x, np.ones(len(x))]).T

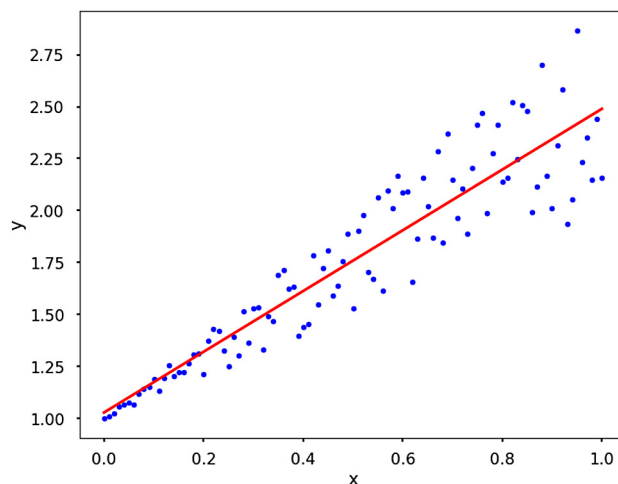
        # turn y into a column vector
        y = y[:, np.newaxis]

In [4]: # Direct least squares regression
        alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T,A)),A.T)),y)
        print(alpha)

[[1.459573 ]
 [1.02952189]]

In [5]: # plot the results
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.plot(x, alpha[0]*x + alpha[1], "r")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()

```



Python has several packages and functions that can perform a least squares regression. These include NumPy, SciPy, statsmodels, and sklearn. Below are several examples of such applications. Feel free to choose the one you like.

16.4.2 USING THE PSEUDO-INVERSE

We mentioned earlier that the matrix $(A^T A)^{-1} A^T$ is called the pseudo-inverse, therefore, we can use the `pinv` function in NumPy to calculate it directly.

```
In [6]: pinv = np.linalg.pinv(A)
        alpha = pinv.dot(y)
        print(alpha)

[[1.459573  ]
 [1.02952189]]
```

16.4.3 USING NUMPY.LINALG.LSTSQ

NumPy has already implemented the least squares methods, so we can just call the function to get a solution. The function will return more data than the solution itself; please check the documentation for details.

```
In [7]: alpha = np.linalg.lstsq(A, y, rcond=None)[0]
        print(alpha)

[[1.459573  ]
 [1.02952189]]
```

16.4.4 USING OPTIMIZE.CURVE_FIT FROM SCIPY

This SciPy function is very powerful. It is not only suitable for linear functions, but many different function forms as well, such as nonlinear functions. Here we will only show the linear example from above. Note that, when using this function, we do not need to turn `y` into a column vector.

```
In [8]: # generate x and y
        x = np.linspace(0, 1, 101)
        y = 1 + x + x * np.random.random(len(x))

In [9]: def func(x, a, b):
        y = a*x + b
        return y

        alpha=optimize.curve_fit(func, xdata=x, ydata=y)[0]
        print(alpha)

[1.44331612 1.0396133 ]
```

16.5 LEAST SQUARES REGRESSION FOR NONLINEAR FUNCTIONS

A least squares regression requires that the estimation function be a linear combination of basis functions. There are some functions that cannot be put in this form, but where a least squares regression is still appropriate.

Introduced below are several ways to deal with nonlinear functions.

- We can accomplish this by taking advantage of the properties of logarithms and transform the nonlinear function into a linear function.
- We can use the `curve_fit` function from `SciPy` to estimate directly the parameters for the nonlinear function using least square.

16.5.1 LOG TRICKS FOR EXPONENTIAL FUNCTIONS

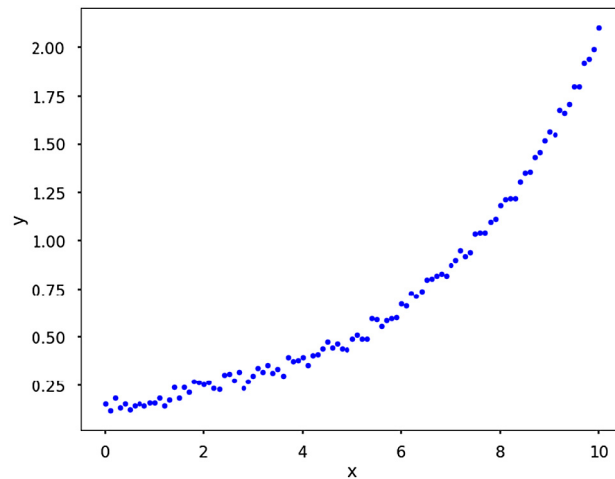
Assume you have a function in the form $\hat{y}(x) = \alpha e^{\beta x}$ and data for x and y , and that you want to perform least squares regression to find α and β . Clearly, the previous set of basis functions (linear) would be inappropriate to describe $\hat{y}(x)$; however, if we take the log of both sides, we get $\log(\hat{y}(x)) = \log(\alpha) + \beta x$. Now, we see that if $\tilde{y}(x) = \log(\hat{y}(x))$ and $\tilde{\alpha} = \log(\alpha)$, then $\tilde{y}(x) = \tilde{\alpha} + \beta x$. Thus, we can perform a least squares regression on the linearized expression to find $\tilde{y}(x)$, $\tilde{\alpha}$, and β , and then recover α by using the expression $\alpha = e^{\tilde{\alpha}}$.

For the example below, we will generate data using $\alpha = 0.1$ and $\beta = 0.3$.

```
In [1]: import numpy as np
        from scipy import optimize
        import matplotlib.pyplot as plt
        plt.style.use("seaborn-poster")

In [2]: # let's generate x and y, and add some noise into y
        x = np.linspace(0, 10, 101)
        y = 0.1*np.exp(0.3*x) + 0.1*np.random.random(len(x))

In [3]: # Let's have a look of the data
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()
```

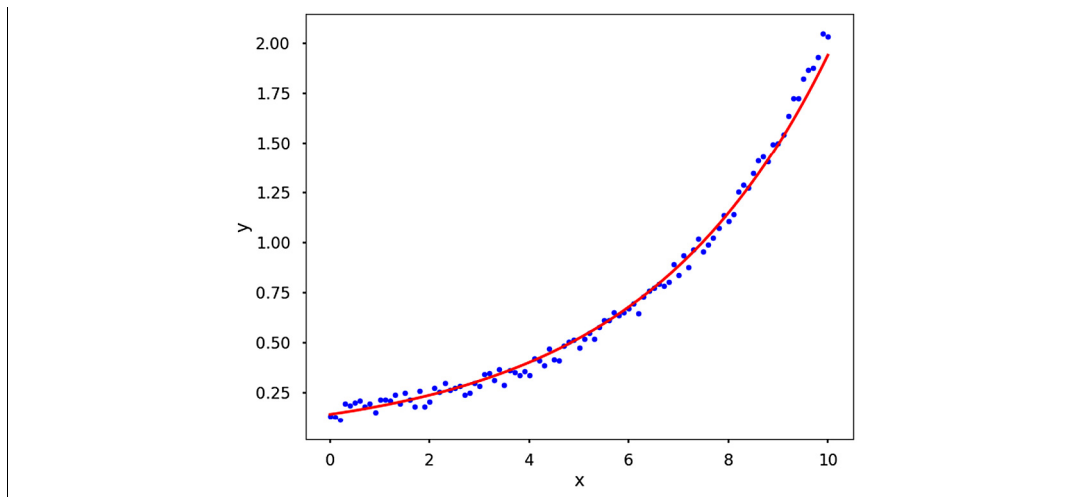



Once the log trick has been applied, we can fit the data.

```
In [4]: A = np.vstack([x, np.ones(len(x))]).T
        beta, log_alpha = np.linalg.lstsq(A, np.log(y), rcond = None)[0]
        alpha = np.exp(log_alpha)
        print(f"alpha={alpha}, beta={beta}")
```

```
alpha=0.13973103064296616, beta=0.26307478591152406
```

```
In [5]: # Let's have a look of the data
        plt.figure(figsize = (10,8))
        plt.plot(x, y, "b.")
        plt.plot(x, alpha*np.exp(beta*x), "r")
        plt.xlabel("x")
        plt.ylabel("y")
        plt.show()
```



16.5.2 LOG TRICKS FOR POWER FUNCTIONS

The power function case is very similar. Assume we have a function in the form $\hat{y}(x) = bx^m$ and data for x and y . Then we can turn this function into a linear form by taking log to both sides: $\log(\hat{y}(x)) = m \log(x) + \log b$, solving this function as a linear regression. Since it is very similar to the above example, we will not devote any more time on this.

16.5.3 POLYNOMIAL REGRESSION

We can also use polynomial and least squares to fit a nonlinear function. Previously, we had our functions all in linear form, that is, $y = ax + b$. But “polynomials” are functions with the following form:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

where $a_n, a_{n-1}, \dots, a_2, a_1, a_0$ are the real number coefficients, and n , a non-negative integer, is the **order** or **degree** of the polynomial. If we have a set of data points, we can use different orders of polynomials to fit it. The coefficients of the polynomials can be estimated using the least squares method as before, i.e., minimizing the error between the real data and the polynomial fitting results.

In Python, we can use `numpy.polyfit` to obtain the coefficients of different order polynomials with the least squares. With the coefficients, we can get the specific values using `numpy.polyval`. Below is an example of how to perform this in Python.

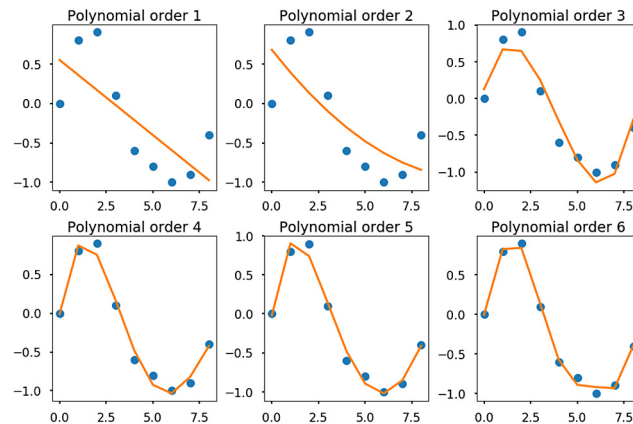
```
In [6]: x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
        y_d=np.array([0,0.8,0.9,0.1,-0.6,-0.8,-1,-0.9,-0.4])

        plt.figure(figsize = (12, 8))
        for i in range(1, 7):
```

```

# get the polynomial coefficients
y_est = np.polyfit(x_d, y_d, i)
plt.subplot(2,3,i)
plt.plot(x_d, y_d, "o")
# evaluate the values for a polynomial
plt.plot(x_d, np.polyval(y_est, x_d))
plt.title(f"Polynomial order {i}")
plt.tight_layout()
plt.show()

```



The figure above shows that we can use different orders of polynomials to fit the same data. The higher the order, the more flexible the data curve required to fit the data. But what order to use is not a simple question, it depends on the specific problems in science and engineering.

16.5.4 USING OPTIMIZE.CURVE_FIT FROM SCIPY

We can use the `curve_fit` function to fit any form function and estimate its parameters. We can solve the above problem using the `curve_fit` function as follows:

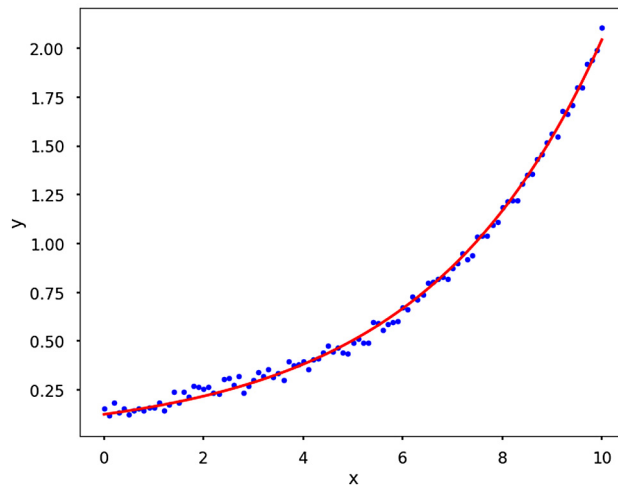
```

In [7]: # let's define the function form
def func(x, a, b):
    y = a*np.exp(b*x)
    return y
alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f"alpha={alpha}, beta={beta}")

alpha=0.12663549356730994, beta=0.27760076897453045

```

```
In [8]: # Let's have a look of the data
plt.figure(figsize = (10,8))
plt.plot(x, y, "b.")
plt.plot(x, alpha*np.exp(beta*x), "r")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



16.6 SUMMARY AND PROBLEMS

16.6.1 SUMMARY

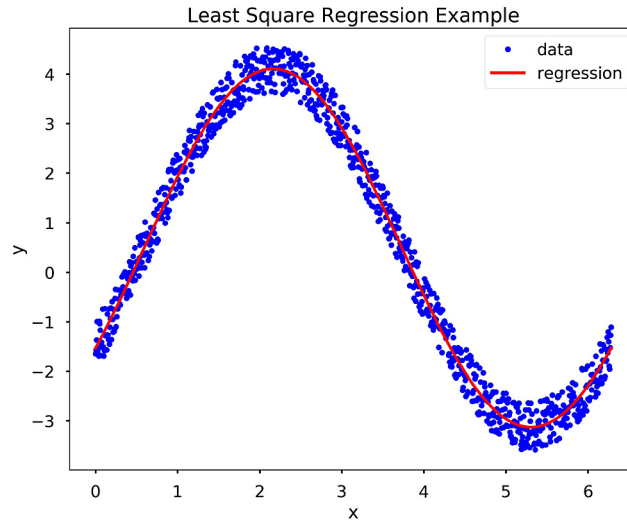
1. Mathematical models are used to understand, predict, and control engineering systems. These models consist of parameters that govern the way the model behaves.
2. Given a set of experimental data, a least squares regression is a method of finding a set of model parameters that fits the data well. That is, it minimizes the squared error between the model, or estimation function, and the data points.
3. In a linear least squares regression, the estimation function must be a linear combination of linearly independent basis functions.
4. The set of parameters β can be determined by the least squares equation $\beta = (A^T A)^{-1} A^T Y$, where the j th column of A is the j th basis function evaluated at each X data point.
5. To estimate a nonlinear function, we transform it into a linear estimation function or use directly a least squares regression to solve the nonlinear function using `curve_fit` from SciPy.

16.6.2 PROBLEMS

1. Repeat the multivariate calculus derivation of the least squares regression formula for an estimation function $\hat{y}(x) = ax^2 + bx + c$, where a , b , and c are the parameters.
2. Write a function `my_ls_params(f, x, y)` where x and y are arrays of the same size containing experimental data, and f is a list with each element a function object to a basis vector of the estimation function. The output argument, `beta`, should be an array of the parameters of the least squares regression for x , y , and f .
3. Write a function `my_func_fit(x, y)` where x and y are column vectors of the same size containing experimental data, and the function returns `alpha` and `beta` are the parameters of the estimation function $\hat{y}(x) = \alpha x^\beta$.
4. Given four data points (x_i, y_i) and the parameters for a cubic polynomial $\hat{y}(x) = ax^3 + bx^2 + cx + d$, what will be the total error associated with the estimation function $\hat{y}(x)$? Can we place another data point (x, y) such that no additional error is incurred for the estimation function?
5. Write a function `my_lin_regression(f, x, y)` where f is a list containing function objects to basis functions, and x and y are arrays containing noisy data. Assume that x and y are the same size. Let an estimation function for the data contained in x and y be defined as $\hat{y}(x) = \beta(1) \cdot f_1(x) + \beta(2) \cdot f_2(x) + \dots + \beta(n) \cdot f_n(x)$, where n is the length of f . Your function should compute `beta` according to the least squares regression formula.
Test Case: Note that your solution may vary by a little bit, depending on the random numbers generated.

```
x = np.linspace(0, 2*np.pi, 1000)
y = 3*np.sin(x) - 2*np.cos(x) + np.random.random(len(x))
f = [np.sin, np.cos]
beta = my_lin_regression(f, x, y)

plt.figure(figsize = (10,8))
plt.plot(x,y,"b.", label = "data")
plt.plot(x, beta[0]*f[0](x)+beta[1]*f[1](x)+beta[2], "r", label="regression")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Least Square Regression Example")
plt.legend()
plt.show()
```



6. Write a function `my_exp_regression(x,y)` where `x` and `y` are arrays of the same size. Let an estimation function for the data contained in `x` and `y` be defined as $\hat{y}(x) = \alpha e^{\beta x}$. Your function should compute α and β to solve the least squares regression formula. Test Cases: Note that your solution may vary slightly from the test case, depending on the random numbers generated.

```
x = np.linspace(0, 1, 1000)
y = 2*np.exp(-0.5*x) + 0.25*np.random.random(len(x))

alpha, beta = my_exp_regression(x, y)

plt.figure(figsize = (10,8))
plt.plot(x,y,"b.", label = "data")
plt.plot(x, alpha*np.exp(beta*x), "r", label="regression")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Least Square Regression on Exponential Model")
plt.legend()
plt.show()
```

